전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계



6주차 ROS2 프로그래밍 #4

3차시 TF2 (3)

* 본 영상은 무단 배포할 수 없습니다.

학습내용

1 TF2 실습 (3)

학습목표

🎯 실습을 통해 TF2를 다룰 수 있다.

TF2 실습 (3)

TF2 실습 - 파이썬

Broadcaster 작성

터미널 입력

```
$ cd ~/ros2_ws/src/ros2_tf/ros2_tf
$ gedit tf2_broadcaster.py
```

TF2 실습 - 파이썬

Broadcaster 작성(tf2_broadcaster.py)

```
from geometry_msgs.msg import TransformStamped
import rclpy
from rclpy.node import Node
from tf2_ros import TransformBroadcaster
from turtlesim.msg import Pose
from tf_transformations import quaternion_from_euler
```

TF2 실습 - 파이썬

Broadcaster 작성(tf2_broadcaster.py)

```
class DyrosBroadcaster(Node):
    def __init__(self):
        super().__init__("tf2_broadcaster")

        self.turtlename = self.declare_parameter(
            'turtlename', 'turtle'
        ).get_parameter_value().string_value

        self.broadcaster = TransformBroadcaster(self)

        self.subscription = self.create_subscription(
            Pose,
            f'/{self.turtlename}/pose',
            self.handle_turtle_pose,
            1
        )
        self.subscription
```

TF2 실습 - 파이썬

Broadcaster 작성(tf2_broadcaster.py)

```
def handle_turtle_pose(self, msg):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'world'
        t.child_frame_id = self.turtlename

        t.transform.translation.x = msg.x
        t.transform.translation.y = msg.y
        t.transform.translation.z = 0.0


        quat = quaternion_from_euler(0, 0, msg.theta)
        t.transform.rotation.x = quat[0]
        t.transform.rotation.y = quat[1]
        t.transform.rotation.z = quat[2]
        t.transform.rotation.w = quat[3]

        self.broadcaster.sendTransform(t)
```

TF2 실습 - 파이썬

Broadcaster 작성(tf2_broadcaster.py)

```
def main():
    rclpy.init()
    node = DyrosBroadcaster()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

TF2 실습 - 파이썬

Broadcaster Launch 작성

터미널 입력

```
$ cd ~/ros2_ws/src/ros2_tf
$ mkdir launch
$ cd launch
$ gedit tf_launch.py
```

TF2 실습 - 파이썬

Broadcaster Launch 작성(tf_launch.py)

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument

def generate_launch_description():
    return LaunchDescription([
        DeclareLaunchArgument(
            'target_frame', default_value='turtle1',
            description='Target frame name.'
        ),
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='sim'
        ),
        Node(
            package='ros2_tf',
            executable="tf2_broadcaster",
            name="turtle1_tf2_broadcaster",
            parameters=[
                {'turtlename': 'turtle1'}
            ]
        ),
    ])
```

TF2 실습 - 파이썬

Broadcaster 작성

터미널 입력

```
$ cd ~/ros2_ws/src/ros2_tf
$ gedit package.xml
```

package.xml

```
...
  <exec_depend>ros2launch</exec_depend>
...
```

TF2 실습 - 파이썬

Broadcaster 작성

터미널 입력

```
$ cd ~/ros2_ws/src/ros2_tf
$ gedit setup.py
```

TF2 실습 - 파이썬

Broadcaster 작성

setup.py

```
import os
from glob import glob
...
    data_files=[
        ('share/ament_index/resource_index/packages',
            ['resource/' + package_name]),
        ('share/' + package_name, ['package.xml']),
        (os.path.join('share',package_name,'launch'),
        glob(os.path.join('launch','*launch.py'))),
    ],
...
    entry_points={
        'console_scripts': [
        'tf2_static_broadcaster=ros2_tf.tf2_static_broadcaster:main',
        'tf2_broadcaster=ros2_tf.tf2_broadcaster:main',
        'tf2_listener=ros2_tf.tf2_listener:main',
        ],
    },
...
```

TF2 실습 - 파이썬

Broadcaster 작성

터미널 1 입력

```
$ cd ~/ros2_ws
$ colcon build --packages-select ros2_tf
$ ros2 launch ros2_tf tf_launch.py
```

터미널 2 입력

```
$ ros2 run turtlesim turtle_teleop_key
```

터미널 3 입력

```
$ ros2 run tf2_ros tf2_echo world turtle1
```

TF2 실습 - 파이썬

Broadcaster 실행

- 거북이 한 마리가 있는 창
- 터미널 2에서 키보드로 조종
- 터미널 3에서 거북이의 rf 상태 확인

TF2 실습 - 파이썬

Broadcaster 실행



터미널 2 출력

```
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.
'Q' to quit.
```

TF2 실습 - 파이썬

Broadcaster 실행



터미널 3 출력

```
At time 1735217648.42091828
- Translation: [7.223, 4.728, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.149, 0.989]
- Rotation: in RPY (radian) [0.000, 0.000, -0.299]
- Rotation: in RPY (degree) [0.000, 0.000, -17.142]
- Matrix:
  0.956  0.295  0.000  7.223
 -0.295  0.956 -0.000  4.728
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
```

TF2 실습 - 파이썬

Listener 작성

tf2_listener.py

```
$ cd ~/ros2_ws/src/ros2_tf/ros2_tf
$ gedit tf2_listener.py
```

TF2 실습 - 파이썬

Listener 작성(tf2_listener.py)

```
import math
from geometry_msgs.msg import Twist
import rclpy
from rclpy.node import Node
from tf2_ros import TransformException
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener
from turtlesim.srv import Spawn
```

TF2 실습 - 파이썬

Listener 작성(tf2_listener.py)

```
class DyroosListener(Node):
    def __init__(self):
        super().__init__("tf2_listener")

        self.target_frame = self.declare_parameter(
            'target_frame', 'turtle1'
        ).get_parameter_value().string_value

        self.buffer = Buffer()
        self.listener = TransformListener(self.buffer, self)

        self.spawner = self.create_client(Spawn, 'spawn')

        self.turtle_spawning_service_ready = False
        self.turtle_spawned = False

        self.turtle_vel = self.create_publisher(Twist,
                                    'turtle2/cmd_vel', 1)

        self.timer = self.create_timer(1.0, self.on_timer)
```

TF2 실습 - 파이썬

Listener 작성(tf2_listener.py)

```
def on_timer(self):
        from_frame = self.target_frame
        to_frame = 'turtle2'

        if self.turtle_spawning_service_ready:
            if self.turtle_spawned:
                try:
                    t = self.buffer.lookup_transform(
                        to_frame,
                        from_frame,
                        rclpy.time.Time()
                    )
                except TransformException as ex:
                    self.get_logger.info(
                        f'Could not transform {to_frame} to {from_frame}: {ex}')
                    return

                msg = Twist()
                msg.angular.z  = 1.0*math.atan2(t.transform.translation.y,
                                                t.transform.translation.x)
                msg.linear.x = 0.5*math.sqrt(t.transform.translation.x**2+
                                             t.transform.translation.y**2)

                self.turtle_vel.publish(msg)
```

TF2 실습 - 파이썬

Listener 작성(tf2_listener.py)

```
else:
                if self.result.done():
                    self.get_logger().info(
                        f'Successfully spawned {self.result.result().name}'
                    )
                    self.turtle_spawned = True
                else:
                    self.get_logger().info('Spawned is not fisnished')

        else:
            if self.spawner.service_is_ready():
                req = Spawn.Request()
                req.name = "turtle2"
                req.x, req.y, req.theta = float(4), float(2), float(0)

                self.result = self.spawner.call_async(req)
                self.turtle_spawning_service_ready = True
            else:
                self.get_logger().info('Service is not ready')
```

TF2 실습 - 파이썬

Listener 작성(tf2_listener.py)

```
def main():
    rclpy.init()
    node = DyroosListener()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass

    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

TF2 실습 - 파이썬

Listener 작성

터미널 입력

```
$ cd ~/ros2_ws/src/ros2_tf/launch
$ gedit tf_listener_launch.py
```

TF2 실습 - 파이썬

Listener Launch 작성(tf2_listener_launch.py)

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package="turtlesim",
            executable="turtlesim_node",
            name="sim"
        ),
        Node(
            package="ros2_tf",
            executable="tf2_broadcaster",
            name="broadcaster_1",
            parameters=[
                {'turtlename':'turtle1'}
            ]
        ),
```

TF2 실습 - 파이썬

Listener Launch 작성(tf2_listener_launch.py)

```
DeclareLaunchArgument(
            'target_frame', default_value='turtle1',
            description='Traget frame name.'
        ),
        Node(
            package="ros2_tf",
            executable="tf2_broadcaster",
            name="broadcaster_2",
            parameters=[
                {'turtlename':'turtle2'}
            ]
        ),
        Node(
            package="ros2_tf",
            executable="tf2_listener",
            name="listener",
            parameters=[
                {'target_frame':LaunchConfiguration('target_frame')}
            ]
        )
    ])
```

TF2 실습 - 파이썬

Listener Launch 작성

tf2_listener.py

```
$ cd ~/ros2_ws/src/ros2_tf
$ gedit setup.py
```

setup.py

```
...
    entry_points={
        'console_scripts': [
        'tf2_static_broadcaster=ros2_tf.tf2_static_broadcaster:main',
        'tf2_broadcaster=ros2_tf.tf2_broadcaster:main',
        'tf2_listener=ros2_tf.tf2_listener:main',
        ],
    },
...
```

TF2 실습 - 파이썬

Listener 실행

터미널 1 입력

```
$ cd ~/ros2_ws
$ colcon build --packages-select ros2_tf
$ source install/setup.bash

ros2 launch ros2_tf tf_listener_launch.py
```

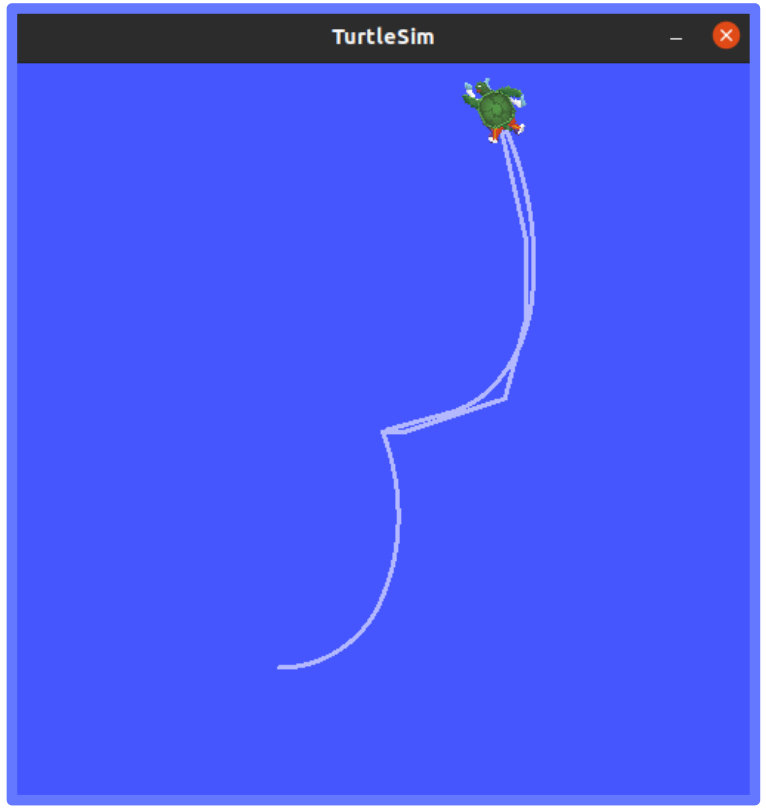
터미널 2 입력

```
ros2 run turtlesim turtle_teleop_key
```

TF2 실습 - 파이썬

Listener 실행

터미널 2를 통해 움직이면 다른 거북이가 계속해서 따라옴



터미널 2 출력

```
Use arrow keys to move the turtle.
Use G|B|V|C|D|E|R|T keys to rotate to absolute orientations. 'F' to cancel a rotation.
'Q' to quit.
```



정리하기

TF2

- TF2를 통해 로봇의 Frame을 기술할 수 있음
  - 로봇의 많은 관절과 링크를 TF2로 표현한다.
  - TF는 Frame과 Frame간의 변환(Transformation)을 의미한다.
  - 변환(Transformation)은 Matrix 형태로 표현 하며, 좌표 변환 실습을 진행한다.